

Fast PDF Layout Segmentation with Transformer-Based Models

Jiajun Shen, Yizhi Zhao, Joy Liu

December 2025, EPFL

Abstract—We model PDF layout segmentation as a sequence tagging task under the HURIDOCs PDF Features framework, which recognizes a series of tokens from PDF pages. Using the DocLayNet-v1.2 Dataset, we propose a lightweight Transformer encoder that predicts token types and segment boundaries, reducing reliance on complex handcrafted features. We compare independent two-step training, cascaded training, and an end-to-end unified labeling approach, and find cascaded training to be the most robust for boundary prediction and final segment and classification quality. Our final model achieves competitive accuracy while keeping computation practical and efficient for large-scale PDF processing.

I. INTRODUCTION

PDF layout segmentation aims to decompose a document page into coherent and semantically meaningful regions, such as titles, paragraphs, and tables. This serves as a fundamental step for downstream document understanding tasks, including optical character recognition (OCR), and information extraction. Unlike plain text, PDF documents encode content in a highly structured yet implicit manner. Logical reading order and semantic structure must be inferred from both textual and layout cues.

Traditional approaches to PDF layout segmentation rely heavily on handcrafted features derived from font properties, spatial coordinates, and heuristic rules [1], [2]. While such methods are computationally efficient, they often require extensive feature engineering and may struggle to generalize across diverse document styles and formats[3]. Recent advances using Transformer-based architectures [4], [5] provide a promising alternative by enabling models to capture long-range contextual dependencies with reduced reliance on manually designed features.

HURIDOCs’s PDF Document Layout Analysis project¹ provides a fast mode choice to accomplish document segmentation with LightGBM models [6], [7]. These models take the features of a single token as input, so they require manual selection and building features for the context. Our goal is to create another computationally fast method by employing a Transformer-based model, while remaining compatible with the original project framework.

Using Transformer-based models allows us to reduce reliance on handcrafted features, particularly those designed to capture contextual information [8]. Moreover, PDF layout segmentation can naturally be formulated as a sequence tagging task, where each page is represented as a variable-length sequence of tokens and is modeled as a coherent structured unit, analogous to a sentence in natural language.

The main contributions of this work are threefold:

- Building upon the PDF Features framework, we propose a lightweight Transformer-based model that formulates PDF layout segmentation as a sequence tagging task.
- We conduct extensive experiments to analyze the impact of different model parameters and design choices, complemented by detailed ablation studies on key components.
- Based on the experimental findings, we train a final model on the full training set, achieving competitive performance in PDF layout segmentation while maintaining computational efficiency.

II. DATASET AND DATA EXTRACTION

The DocLayNet-v1.2 dataset² is the latest version of the DocLayNet dataset [9] on which the HURIDOCs’s LightGBM models are trained. DocLayNet-v1.2 contains more than 69,400 human-annotated PDFs in the training set and about 4,000 in the test set. The annotated segments in the dataset are of eleven possible types of text, each with a bounding box indicating the position and size of the segment. However, the bounding boxes are annotated on images of PDF pages resized to 1024×1024 pixels, so we convert the bounding boxes to the original PDF size to make it compatible with the annotation schema for data extraction.

To convert PDFs into a machine-readable data structure, HURIDOCs provides the PDF Features library³, a Python toolkit for extracting structured token-level representations from PDF documents [10]. The library parses PDF files into XML-based representations and extracts text tokens together with their associated layout and styling information, including bounding box coordinates, font properties, and text styles. Each token corresponds to a small text block (e.g., a line of text) and serves as the basic unit for subsequent modeling.

Based on these extracted features, PDF Features supports token type annotation by defining a set of semantic categories for text. The library provides utilities for loading labeled data, managing token-level annotations, and handling page-wise label assignments. Together, this forms a complete preprocessing pipeline for PDF layout analysis.

III. MODELS AND IMPLEMENTATION

A. Task Formulation

We formulate PDF segmentation as a sequence tagging problem. Given a PDF document, we first apply a parser to obtain an ordered token sequence $\mathbf{x} = (x_1, \dots, x_T)$, where each token corresponds to a small text block (e.g., a line of text) and is associated with both textual content and layout features (e.g., bounding box coordinates). These tokens are automatically recognized using the PDF Features toolkit.

¹<https://github.com/huridocs/pdf-document-layout-analysis>

²<https://huggingface.co/datasets/docling-project/DocLayNet-v1.2>

³<https://github.com/huridocs/pdf-features>

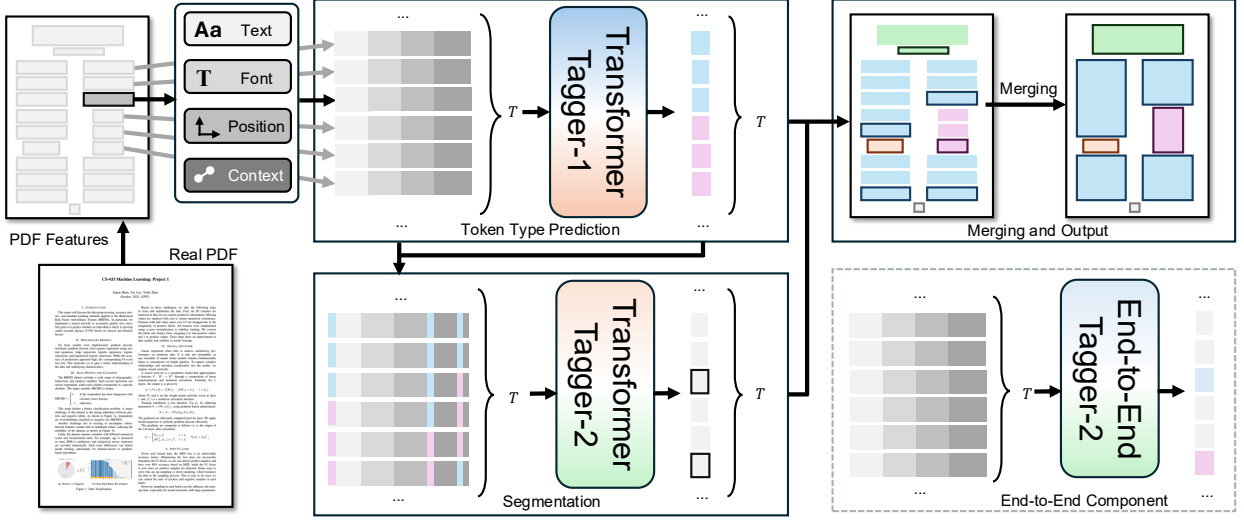


Figure 1: The figure shows the pipeline of segmenting a single PDF page. We first convert a PDF into a sequence of tokens with text and position information, then we extract the metadata of each token to build features. The tokens are regarded as a sequence, fed into the first model, which returns the type prediction. The prediction is integrated into the features for segment boundary prediction. Considering both the token type and the boundary, we can merge them and label them as a certain type.

For each token x_i , the model predicts (i) a token type label $t_i \in \{1, \dots, K\}$ and (ii) a binary segment boundary indicator $b_i \in \{0, 1\}$, where $b_i = 1$ indicates that token x_i ends a segment. Formally, the predictions are given by $t_i = \arg \max_k p(t_i = k | \mathbf{x})$ and $b_i = \arg \max_{c \in \{0, 1\}} p(b_i = c | \mathbf{x})$.

A segment is defined as a maximal contiguous subsequence $\{x_s, \dots, x_e\}$ such that $b_e = 1$ and $b_j = 0$ for all $s \leq j < e$. The segment-level type is then determined by majority voting over token-level type predictions within the segment, i.e., $\hat{t}_{s:e} = \text{mode}(\{t_s, \dots, t_e\})$.

B. Transformer Encoder for Sequence Tagging

Given an input PDF token sequence of a certain page $\mathbf{x} = (x_1, x_2, \dots, x_T)$, where each token x_i is represented by a feature vector encoding textual and layout information, the sequence is mapped to d -dimensional embeddings with positional information:

$$\mathbf{E} = \text{Emb}(\mathbf{x}) \in \mathbb{R}^{T \times d}.$$

The embedding sequence is fed into a Transformer encoder:

$$\mathbf{H} = \text{Encoder}(\mathbf{E}) \in \mathbb{R}^{T \times d},$$

Thus, the encoder preserves the sequence length while producing a contextualized representation of each token.

For token-level prediction, each hidden state $\mathbf{h}_i$ is mapped to tag scores via a shallow neural network:

$$\mathbf{Z} = f_\theta(\mathbf{H}) \in \mathbb{R}^{T \times K}.$$

The label sequence is thus determined by selecting the index with the maximum score at each position:

$$\mathbf{y} = (y_1, y_2, \dots, y_T), \quad y_i = \arg \max_{k \in \{1, \dots, K\}} Z_{i,k}.$$

This formulation is generic and can be instantiated for different prediction tasks by choosing an appropriate label space. Specifically, when $\mathbf{y} = \mathbf{t}$ with $t_i \in \{1, \dots, K\}$, the model performs

token type prediction. When $\mathbf{y} = \mathbf{b}$ with $b_i \in \{0, 1\}$, the same architecture is used for segment boundary detection.

Furthermore, an end-to-end segmentation and labeling formulation can be obtained by defining a unified label set. In this case, we extend the label space to $\{0, 1, \dots, K\}$, where $y_i = 0$ indicates that x_i is not a segment boundary, and $y_i = k \in \{1, \dots, K\}$ indicates that x_i ends a segment of type k . Under this formulation, both the token type and boundary information can be recovered from a single prediction sequence. We show in Figure 3 an example output of the token type task followed by the segmentation task.

C. Implementation

Input: To effectively capture the multi-modal nature of document layouts, we construct a dense feature vector for each token x_i by aggregating information from four distinct modalities: Geometrical Features, Lexical and Statistical Features, Font Features, and Contextual Features. Ablation studies show they are all necessary and details about the features are listed in A.

Model: The Transformer model is implemented with an initial input projection layer comprising a Multi-Layer Perceptron (MLP) with LayerNorm and ReLU activation. This projects the raw feature vector onto the hidden dimension d before adding sinusoidal positional encodings. The encoder consists of a stack of Transformer layers followed by a final MLP classification head.

Training Strategy: The distribution of token classes in document layout datasets is typically highly imbalanced (e.g., *Body Text* tokens vastly outnumber *Section Header* or *Caption* tokens). Standard Cross-Entropy (CE) loss tends to be biased towards majority classes in such scenarios. To mitigate this, we implement a combined objective function that integrates Focal Loss [11].

The loss for a single token with target class y and predicted probability p_t is defined as:

$$\mathcal{L} = \alpha_t (1 - p_t)^\gamma \cdot \text{CE}_{ls}(p_t, y)$$

Table I: Performance Comparison (l =Layers, d =Hidden Dim, ep =Epochs)

Configuration	Token Type Acc (%)	Independent Training		Cascaded Training		End-to-End	
		Segment Boundary	Adjusted Token Type	Segment Boundary	Adjusted Token Type	Segment Boundary	Token Type Acc
LightGBM	87.46	98.84	87.46 (-0.00)	–	–	–	–
$l=6, d=512, \gamma=2, ep=30$	69.28	82.26	69.58 (+0.30)	92.85	68.51 (-0.77)	72.20	68.92
$l=4, d=512, \gamma=2, ep=30$	77.06	90.91	76.82 (-0.24)	93.22	77.46 (+0.40)	78.28	75.01
$l=6, d=256, \gamma=2, ep=30$	71.36	85.81	71.16 (-0.20)	91.82	71.41 (+0.05)	76.19	72.99
$l=6, d=512, \gamma=1, ep=30$	68.77	68.38	68.38 (-0.38)	87.17	68.38 (-0.38)	71.74	67.98
$l=6, d=256, \gamma=2, ep=20$	24.71	84.28	24.71 (-0.00)	90.39	23.22 (-1.49)	71.18	67.39

where $(1 - p_t)^\gamma$ is the modulating factor from Focal Loss that down-weights the loss contributions of easy-to-classify examples (with default focusing parameter $\gamma = 2.0$).

IV. EXPERIMENTS AND RESULTS

In this section, we evaluate the performance of our proposed Transformer-based models on a subset of DocLayNet-v1.2 training sets, which are split into 3910 training samples and 978 validation samples. Both models employ rich geometric and textual features derived from PDF Features. The token type model uses 39 features, and the Segment Boundary model integrates an additional three token-type features (totaling 42) to explicitly model semantic continuity between neighboring tokens. We first study the impact of key hyperparameters, then compare different model architectures (Independent, Cascaded, and End-to-End).

A. Different Parameters

We conducted a comprehensive hyperparameter analysis to identify the optimal configuration for our Transformer-based architecture.

Initial experiments indicated that larger models were computationally expensive. Specifically, models with $\text{Layer_num} > 8$ or $\text{Hidden_dim} > 512$ were unstable. Consequently, we focused our exploration on a search space where $\text{Layer_num} \leq 6$ and $\text{Hidden_dim} \leq 512$. Due to limited resources, we explore the impact of different parameters including Layer_num , Hidden_dim , weighting γ of Focal Loss, and Epoch, on the subset of DocLayNet-v1.2 training sets.

B. Different Strategies

We explored three distinct architectural strategies to solve the layout analysis task:

- 1) **Two steps (Independent):** The first model predicts token types (11 classes). The second predicts binary segment boundaries, trained using **true** token types as input.
- 2) **Two steps (Cascaded):** The second model is trained using the **noisy** token type predictions from the first model, simulating a real inference pipeline.
- 3) **End-to-End:** A single model predicts a combined 12-class space (11 boundary types + 1 trivial internal token type).

C. Evaluation

During segmentation, all tokens within a predicted segment are assigned the same label, typically using majority voting. As a result, segmentation may alter the labels of a small portion of tokens. Therefore, in addition to the original token-type accuracy, we also report the adjusted token-type accuracy after segmentation to better reflect the final prediction quality. The results are shown in Table I.

D. Analysis and General Rule

Comparing the approaches reveals that cascaded training reaches higher segmenting performance, as it learns to handle the noise from the previous model, and the segmentation model tends to make the boundary more accurate. In contrast, independent training is highly dependent on the performance of the preceding model. Errors introduced in the first step are not effectively mitigated and may even degrade boundary accuracy. The End-to-End model demonstrates reasonable performance in token type classification, but its segmentation performance remains substantially lower.

Therefore, our final design choice is to separate token classification and segmentation into two stages and adopt cascaded training to ensure segmentation quality. While the result of LightGBM is significantly better in these experiments, we still notice:

- The implementation of LightGBM implies that increasing the dataset size will also lead to an increase in model size. Therefore, while we can train our Transformer model on a larger dataset without increasing its size, transferring LightGBM to a larger training set would incur a high computational cost.
- Since LightGBM requires prediction at the token level, it cannot perform parallel computation over an entire page like Transformers, which results in increased inference time.

As shown in Section VI, our model, when trained on the full training set, achieves comparable performance while maintaining a smaller size and faster inference speed.

V. ABLATION STUDIES

In this section, we conduct ablation studies on the same subset of DocLayNet-v1.2 training set to analyze the impact of two key designs introduced to address the issues observed in our initial experiments.

We identified two major issues in our preliminary experiments. The first issue is severe label imbalance. The majority of

tokens belong to the text and table classes, while other layout categories are far less represented. The second issue is that lightweight models remain difficult to train effectively without explicit contextual features, despite Transformer-based models capturing contextual information implicitly.

To address these issues, we introduce two modifications:

- 1) Focal Loss combined with label smoothing to mitigate class imbalance, and
- 2) Relational contextual features from the previous and next tokens to enrich the input representation.

We evaluate the contribution of each component through ablation experiments. The results are shown in Table II and Figure 2.

Table II: Ablation Study Results.

Components		Accuracy (%)		
w/ Focal Loss	w/ Context	Token Type	Segment Boundary	Adjusted Token Type
✓	✓	69.28	92.85	68.51
×	✓	50.68	93.07	49.49
✓	×	65.65	86.58	64.64
×	×	68.09	88.83	67.53

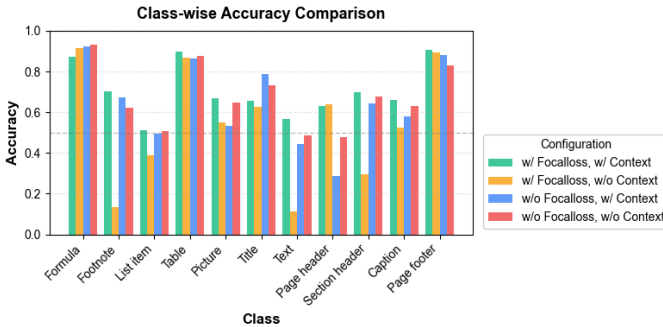


Figure 2: Class-wise accuracy comparison in ablation study. The inclusion of Focal Loss (green) significantly recovers the performance of less represented classes (e.g., Footnote, Formula), which are otherwise ignored by the model (orange) due to the dominance of Text class labels.

We can see from the results that including both Focal Loss and contextual features achieves the best token type accuracy and a high segment boundary accuracy. When used with contextual features, Focal loss is useful for handling class imbalance, but has minimal impact on boundary detection. When contextual features are removed, token type accuracy drops moderately while segment boundary accuracy significantly decreases. This demonstrates that contextual features are essential for both token classification and boundary detection, especially for boundary detection. Through Figure 2, we can see that when Focal Loss is removed, minority classes suffer heavy degradation, which confirming that Focal Loss prevents overfitting to majority classes.

Though Transformer models allow us to reduce many features, but we also conduct experiments to confirm necessary features. These results can be seen in Table III.

Table III: Ablation Study Results on other features

Features	Token Type Acc (%)	Segment Boundary	Adjusted Token Type
All	69.28	92.85	68.51
w/o content	66.82	94.40	66.61
w/o font	66.04	78.68	64.64
w/o position	56.38	77.72	63.19

VI. FINAL MODEL TRAINING AND RESULTS

Given the advantages of cascaded training, we ultimately adopt this strategy to train the token type predictor on the full training set, using the best hyperparameters identified in previous experiments.

Finally, our models reach 90.39% token type accuracy and 94.30% segmenting boundary accuracy. The models are of size 26.3MB and 23.6MB each, with the token type predictor being 76.5% smaller than the LightGBM model (111.7MB) trained in IV. Due to the ability to tag tokens of one page in parallel, this model can predict each page within 0.015s on CPU, 81.3% faster than the baseline of 0.080s, demonstrating the potential for large-scale PDF processing. An example of token type prediction and segmentation generated by our models can be found in Appendix C.

VII. CONCLUSION

We presented a lightweight Transformer-based approach for PDF layout segmentation, formulating the task as a sequence tagging problem with cascaded training for improved robustness. Compared to LightGBM-based pipelines, our method achieves comparable segmentation and classification performance while significantly reducing model size and inference time. By enabling page-level parallel processing and avoiding model growth with increasing data volume, the proposed approach is particularly well-suited for large-scale PDF processing scenarios, where computational efficiency and scalability are critical.

VIII. LIMITATIONS

Due to computational constraints, we were unable to conduct exhaustive experiments on the full training set. Performing experiments and ablation studies on a subset may yield different results, particularly since the model is expected to scale proportionally with the size of the dataset. While our final results also show a significant improvement due to the scaling of data, it could still be suboptimal.

Our evaluation metrics are currently based on token-level label accuracy. However, these metrics do not necessarily reflect human perception of PDF layout quality. Metrics that better capture the overall structure, such as the mean Intersection over Union (mIoU) of different segments, would provide a more meaningful assessment and enable more reliable comparison between different segmentation pipelines.

IX. ACKNOWLEDGMENT

We would like to thank EPFL for providing the opportunity through the ML4S program. We are also grateful to HURI-DOCS for their support and collaboration, as well as to Gabriel Piles, Jaume Cardona, and Ali Altıparmak for their valuable

guidance and support throughout the technical development and project workflow.

REFERENCES

- [1] A. Corbelli, L. Baraldi, C. Grana, and R. Cucchiara, “Historical document digitization through layout analysis and deep content classification,” vol. 0, 2016, Conference paper, p. 4077 – 4082, cited by: 13; All Open Access, Green Open Access. [Online]. Available: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85019108141&doi=10.1109%2FICPR.2016.7900272&partnerID=40&md5=b82347e3a1e032e014b5a885733962de>
- [2] C. Xu, Z. Tang, X. Tao, and C. Shi, “Integration of text information and graphic composite for pdf document analysis,” *Communications in Computer and Information Science*, vol. 333 CCIS, p. 13 – 22, 2012, cited by: 0. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84869225644&doi=10.1007%2F978-3-642-34456-5_2&partnerID=40&md5=030fb07551609dfd64c51b4fb8f9957a
- [3] Z. Ibrahim, D. Isa, and R. Rajkumar, “Text and non-text segmentation and classification from document images,” vol. 1, 2008, Conference paper, p. 973 – 976, cited by: 9. [Online]. Available: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-61849083563&doi=10.1109%2FCSSE.2008.1516&partnerID=40&md5=853b810270d603c0e94c5e569ea70cf5>
- [4] P. Pillai and P. Mangsuli, “Document layout analysis using detection transformers,” in *Abu Dhabi International Petroleum Exhibition and Conference*. SPE, 2021, p. D031S102R001.
- [5] J. Wang, K. Hu, and Q. Huo, “Dlaformer: An end-to-end transformer for document layout analysis,” in *International Conference on Document Analysis and Recognition*. Springer, 2024, pp. 40–57.
- [6] HURIDOCS, “Pdf document layout analysis,” <https://github.com/huridocs/pdf-document-layout-analysis>, 2025, accessed: 2025-12-18.
- [7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [9] B. Pfitzmann, C. Auer, M. Dolfi, A. Nassar, and P. Staar, “Doclaynet: A large humanannotated dataset for document-layout analysis (2022),” URL: <https://arxiv.org/abs/2206>, vol. 1062, p. 17.
- [10] HURIDOCS, “pdf-features: A python library for extracting and analyzing features from pdf documents,” <https://github.com/huridocs/pdf-features>, 2025, accessed: 2025-12-18.
- [11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2980–2988.

APPENDIX

A. Impelmentation Details

Our FeatureExtractor module generates these features as follows:

- **Geometrical Features:** We utilize the bounding box coordinates $(x_{min}, y_{min}, x_{max}, y_{max})$ normalized by the page width and height to ensure scale invariance. and relative width, height, aspect ratio, and the center coordinates of the token. Positional flags are also included to indicate if a token resides in specific page regions (e.g., margins typically reserved for headers or footers).
- **Lexical and Statistical Features:** We extract statistical properties of the text content, including the length of the string, the number of words, and the density of uppercase characters, digits, and whitespace. Binary indicators are used to flag the presence of punctuation, full capitalization, or numeric content, which are strong signals for identifying headers and page numbers.
- **Font Features:** Distinctive font styles are critical for layout analysis. We incorporate the font ID and font size directly. To handle the high variance in font sizes across different documents, we also include the logarithm of the font size ($\log(\text{size} + 1)$).
- **Contextual Features:** Since layout analysis depends heavily on local context, we explicitly encode the relationship between the current token and its immediate neighbors (preceding and succeeding tokens). This includes the horizontal and vertical distances between bounding boxes and the difference in font sizes. A binary “same-line” feature is computed based on the vertical alignment to explicitly capture line structures.

B. Ethical Risks

One ethical risk of this project is the vulnerability and misuse of personal data contained in the PDFs processed by our model. Documents may include sensitive information such as names, medical records, and personal details. In the context of HURIDOCS, processing PDFs with the goal of identifying human rights violations must be treated carefully to protect individuals mentioned in the documents.

The stakeholders most impacted by this risk are the individuals whose personal information appears in the documents. Possible negative impacts include privacy violations, causing harm to reputation, and, in severe cases, physical or legal risks for individuals mentioned in sensitive reports. While our model does not explicitly extract textual content beyond layout structure, improving segmentation quality can improve information extraction, thus lowering the barrier to misuse. The severity of this is tied to the nature of the documents involved, and misuse depends on downstream applications rather than the model alone.

To study the potential of this risk, we examined the DocLayNet-v1.2 dataset to confirm that it consists of publicly available documents and does not include explicit annotations of sensitive information. Our assessment is entirely qualitative and contextual. It may be possible to improve this assessment by introducing metrics to quantify privacy risk.

Our model is constrained to operate only on token-level layout, font, and geometric features. We did not train language

models or embeddings that learn textual content or semantic information. Additionally, our final model is treated as an infrastructural component rather than an end-to-end user tool. We actively discourage deployment on private or non-consensual document collections. A remaining barrier is that downstream misuse cannot be fully prevented at the model level; mitigating this requires governance, access control, and responsible deployment practices beyond the scope of this project.

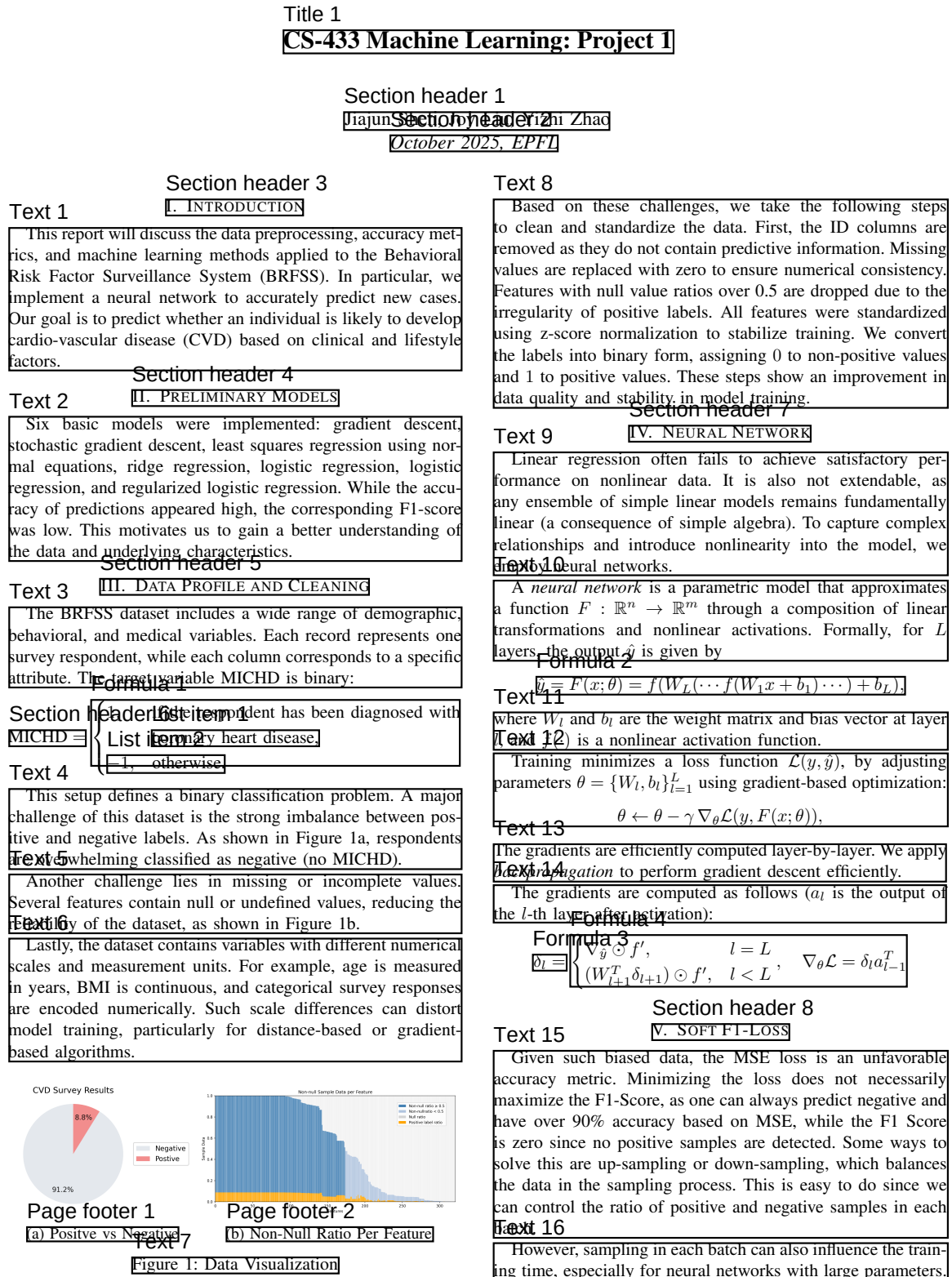


Figure 3: Example of segmentation using our model. The PDF is our first report of this course.